

La forma más cómoda y potente de hacer squash “por grupos” (por ejemplo todos los commits que contengan audiencia o lecturas) es combinar:

1. `git rebase -i`
2. un pequeño script/alias que prepare automáticamente la lista

Método manual rápido (el estándar)

Si quieres unir los últimos 8 commits: `git rebase -i HEAD~8`

Te abrirá algo así:

`pick a1b2c3 commit 1`

`pick d4e5f6 commit 2`

`pick g7h8i9 commit 3`

Cambias los que quieres unir:

`pick a1b2c3 commit 1`

`squash d4e5f6 commit 2`

`squash g7h8i9 commit 3`

Guardas y Git los combina.

Opción MUY útil: autosquash con fixup

Git tiene una funcionalidad brutal llamada **--autosquash**. Probablemente es lo mejor para trabajo diario porque:

- no editas manualmente
- no buscas hashes
- Git entiende relaciones entre commits
- funciona genial para “commits temáticos”

Flujo recomendado

1. Commit principal `git commit -m "audiencia"`

2. Commits relacionados

En vez de: `git commit -m "cambio css audiencia"`

usas: `git commit --fixup :/"audiencia"`

o: `git commit --fixup <hash>`

Git crea algo así: `fixup! audiencia`

3. Cuando quieras fusionarlos todos

`git rebase -i --autosquash HEAD~20`

Git automáticamente moverá y marcará los siguientes quedando todo unido:

pick xxx audiencia
fixup yyy fixup! audiencia
fixup zzz fixup! audiencia

Automatización total por patrón

Puedes hacer un alias. Ejemplo:

```
git config --global alias.squashby '!f() {\n  git log --oneline --grep="$1"; \n}; f'
```

Uso: `git squashby audiencia`

Te lista commits relacionados.

```
correomatic on 7 correomatic [58] via 2 v22.13.0
• > git squashby audiencia
4ceb383 (HEAD -> correomatic) audiencia 13/05/26
4672708 mejroas audiencias
fed2635 reorganización de audiencias y mejora
b97f217 audiencia 29/04/26
c455434 nueva audiencia 08/04/26
a7f32b7 nueva audiencia 01/04/26
971a5fd foto actual con generacion de audiencias en we
0ee3f9d correcciones audiencias
07e7761 audiencia papa 18/03/26
8d56866 index donde debe estar en public/audiencias
92bbe25 versión operativa y 100% funcional para públic
a0556a5 audiencias automatizadas, fuente editorial úni
e6419f6 generador audiencias
8621cf8 audiencias
6998338 script audiencia
```

O versión avanzada: squash automático

Ejemplo:

```
#!/bin/bash
PATTERN=$1
BASE=$(git merge-base HEAD main)
COMMITTS=$(git log --reverse --format="%H %s" $BASE..HEAD | grep "$PATTERN" |
awk '{print $1}')
FIRST=$(echo "$COMMITTS" | head -n1)
echo "Primer commit: $FIRST"
echo "Squasheando commits relacionados con: $PATTERN"
git rebase -i $FIRST^
```

Luego solo cambias a squash los deseados.

Diferencia importante: git reflog vs git log

git reflog parece más sucio porque muestra:

- rebases
- resets
- checkouts
- squash intermedios
- movimientos de HEAD

git log muestra el historial REAL de commits (ahí vemos si quedó limpio)

git log --online --graph / **git log --online --decorate** / **git log --online -20**

```
> git reflog
4ceb383 (HEAD -> correomatic) HEAD@{0}: rebase (finish): returning to refs/heads/correomatic
4ceb383 (HEAD -> correomatic) HEAD@{1}: rebase (squash): audiencia 13/05/26
1bb0951 HEAD@{2}: rebase (squash): # This is a combination of 7 commits.
9ee0d01 HEAD@{3}: rebase (squash): # This is a combination of 6 commits.
1118858 HEAD@{4}: rebase (squash): # This is a combination of 5 commits.
f4223ab HEAD@{5}: rebase (squash): correomatic on v22.13.0 via v22.13.0 took 3m3s
8633275 HEAD@{6}: rebase (squash): git logall
79962bf HEAD@{7}: rebase (squash): * 4ceb383 (HEAD -> correomatic) audiencia 13/05/26
2d2239e HEAD@{8}: rebase (start): c * 17051e8 (origin/correomatic) imagenes
17051e8 (origin/correomatic) HEAD@{9}: * 7a2beee lect anunc
7a2beee HEAD@{10}: commit: lect anu * cf62c3a 16/05
cf62c3a HEAD@{11}: commit: 16/05 * 8a394bd san isidro y viernes 6 Pascua
8a394bd HEAD@{12}: commit: san isid * 5e5968c aduciencias fix
5e5968c HEAD@{13}: commit: aducienc * be14077 audiencias
be14077 HEAD@{14}: commit: audienci * 39919d0 lecturas 14/05 correomatic on v22.13.0 via v22.13.0 took 3m3s
39919d0 HEAD@{15}: commit: lecturas * 2d2239e audiencia 13/05/26
2d2239e HEAD@{16}: commit: audienci /
3c5dd33 HEAD@{17}: commit: lecturas * 3c5dd33 lecturas
449f959 HEAD@{18}: commit: ABC-6-se * 449f959 ABC-6-semana-Pascua
c557f0e HEAD@{19}: commit: actualiz * c557f0e actualizado roadmap
be6eceb HEAD@{20}: commit: document * be6eceb documentacion actualizad
ac80e41 HEAD@{21}: commit: plantill * ac80e41 plantilla nueva fuera de
91108ed HEAD@{22}: commit: guia red * 91108ed guia redeploy functi
b9021f5 HEAD@{23}: commit: fix: for * b9021f5 fix: forzar redeploy de
87d63c6 HEAD@{24}: commit: Forzar r * 87d63c6 Forzar redeploy de la fu
4831bc6 HEAD@{25}: commit: lecturas * 4831bc6 lecturas 11/05
15ac7cf HEAD@{26}: commit: feat(ui) * 15ac7cf feat(ui): rediseño compl
3167fbd HEAD@{27}: commit: actualiz * 3167fbd actualizado roadmpa-corr
47c4752 HEAD@{28}: commit: intento * 47c4752 intento fix error prod.
c861659 HEAD@{29}: commit: fix: for * c861659 fix: forzar redeploy de
6d84b8f HEAD@{30}: commit: fix: net * 6d84b8f fix: netlify.toml con bu
d23179e HEAD@{31}: commit: fix: net * d23179e fix: netlify.toml sin bu
```

Vemos mensajes como # This is a combination of 7 commits.

Eso es parte del proceso interno del squash. Git:

- crea commits temporales
- reescribe hashes
- mueve HEAD varias veces

El reflog registra TODO. Para ver “limpio” usamos **git log --online --graph o gitk**, no **git reflog**

A tener en cuenta sobre squash

Una vez squasheas:

- los hashes cambian
- el historial anterior desaparece del log
- PERO reflog conserva referencias temporales unos días

Eso es bueno:

- puedes recuperar cosas
- puedes deshacer errores

Para dejar el historial impecable

El flujo ideal es: **Durante desarrollo** muchos commits: (fix css, ajuste, debug, prueba)

Antes de push

```
git rebase -i HEAD~15
```

y dejamos:

feat: audiencias

feat: lecturas anunciacion

fix: emails worker

Borrar un alias `git config --global --unset alias.logod`

Saber los alias que tienes configurados `git config --global --get-regexp alias`

Ver reflog compacto `git config --global alias.rl "reflog --oneline"`

alias.**logall** log --oneline --graph --all --decorate

alias.**logod** log --oneline --decorate

alias.**lg** log --graph --oneline --decorate --all

alias.**gg** log --graph --all --decorate --date=relative --

pretty=format:'%C(auto)%h%C(reset) %C(bold blue)%an%C(reset) %C(dim white)- %ar%C(reset)%C(auto)%d%C(reset)%n%C(white)%s%C(reset)%n'

alias.**l** log --oneline --graph --decorate --all

alias.**ld** log --graph --decorate --all -p

alias.**st** status

alias.**co** checkout

Ver últimas ramas usadas `alias.br branch`

Commit rápido alias.**cm** commit -m

Últimos 10 commits alias.**last10** "log -10 --oneline"

Lista commits relacionados con un patrón/palabra (git squashby audiencia)

```
alias.squashby !f() { \  
  git log --oneline --grep="$1"; \  
}; f
```

Buscar commits (git find audiencia) alias.**find** log --all --oneline --decorate --graph
-grep

Buscar commits + archivos modificados (git findfiles audiencia)

alias.**findfiles** log --all --stat --oneline -grep

Configurar un alias

```
git config --global alias.logod "log --oneline --decorate"
```

Después de un rebase si intentas pull sin hacer nada más no te deja, GIT te protege.

```
correomatic on 1/ correomatic [$0] via v22.13.0  
➤ git push  
To https://github.com/andune55/correomatic.git  
! [rejected]        correomatic -> correomatic (non-fast-forward)  
error: failed to push some refs to 'https://github.com/andune55/correomatic.git'  
hint: Updates were rejected because the tip of your current branch is behind  
hint: its remote counterpart. If you want to integrate the remote changes,  
hint: use 'git pull' before pushing again.  
hint: See the 'Note about fast-forwards' in 'git push --help' for details.  
❖  
correomatic on 1/ correomatic [$0] via v22.13.0  
➤
```

es normal porque he reescrito mi rama local

Qué significa esto

Your branch and 'origin/correomatic' have diverged

Significa:

Local

Tu rama ahora tiene: A---B---C

porque el squash creó commits NUEVOS.

Remote (GitHub)

GitHub sigue teniendo: A---X---Y---Z

los commits antiguos pre-squash.

Por qué Git rechaza el push → Git intenta protegerte.

El push normal solo funciona si el remoto es:

- igual
- o más antiguo

Pero aquí:

- tu historia cambió
- hashes distintos
- commits antiguos desaparecieron

Entonces Git dice: non-fast-forward

EL ERROR MÁS COMÚN AQUÍ

NO hagas esto: `git pull`

Porque mezclarías:

- la historia vieja
- con la nueva squasheada

y te quedaría un caos.

El flujo correcto tras squash/rebase

1. Verifica que TU historial está bien

`git lg o: git log --oneline --graph`

Comprueba:

- commits limpios
 - squash correcto
-

2. Sobrescribe el remoto

Debes hacer:

`git push --force-with-lease`

NO `--force` a secas.

Por qué `--force-with-lease`

Es la versión segura.

Protege si alguien subió algo nuevo mientras tanto.

Qué hará exactamente

GitHub cambiará:

ANTES: A---X---Y---Z

DESPUÉS: A---B---C

y el historial quedará limpio.

Tu caso concreto

Esto: have 1 and 8 different commits each
significa:

- tú tienes 1 commit nuevo squasheado
- remoto tiene 8 commits antiguos

Totalmente coherente.

Flujo profesional recomendado

Desarrollo normal

commit

commit

commit

commit

Limpieza antes de push

`git rebase -i HEAD~10`

Push tras reescribir historia

`git push --force-with-lease`

Regla MUY importante

Haz esto SOLO si:

- ✓ trabajas tú solo en esa rama o
- ✓ nadie más basa trabajo en esos commits

Si trabajas en equipo

Entonces:

- evitar force push en ramas compartidas
- usar feature branches
- squash al mergear PR

Resumen corto

Tu situación es NORMAL tras squash.

Ahora mismo: **Correcto:** `git push --force-with-lease`

Incorrecto: `git pull`

porque rompería la limpieza del historial.